

SNAP UP - INTERN IMPLEMENTATION & TESTING GUIDE

Checkout, staff approval, inventory, barcode scanning, and database synchronisation

Purpose

This is the implementation and testing brief for the Snap Up checkout changes. The goal is to make payment, staff approval, bill generation, inventory reduction, cart scanning, and database synchronisation reliable and auditable.

The intern should implement the required changes, test every scenario in this guide, and send a final test report. Do not make production database changes until the project owner approves them.

1. Desired checkout state machine

A customer scans products into a temporary cart. Scanning a barcode may add a new cart line or increase the quantity of its existing line. It must not create a bill, purchase record, or inventory update.

Customer selects payment and completes the payment-gateway step. Payment success alone must create only a `PENDING_STAFF_APPROVAL` payment/order record. It must not generate a bill, add anything to bill history, or reduce inventory.

Staff opens the pending transaction in the staff portal and verifies it. Only the staff Proceed / Paid acknowledgement completes the sale. In one database transaction: validate stock, reduce available_qty, create the final bill and bill items, mark payment/order `APPROVED` or `COMPLETED`, and write the result to bill history.

After this commit succeeds, notify the customer app. It should move to a new Bill / Receipt page and show the approved bill. Before approval it must show a pending status, never a final bill.

2. Payment gateway requirement

Integrate the selected gateway using its server-side API and webhook/signature verification. Never trust a payment-success value sent directly by the customer app.

Use idempotency keys and gateway transaction IDs. A repeat webhook, page refresh, or staff double-click must not create two bills or deduct stock twice.

Recommended states: `CART`, `PAYMENT_PENDING`, `PAYMENT_RECEIVED_PENDING_STAFF`, `APPROVED_COMPLETED`, `DECLINED_OR_CANCELLED`, and `REFUNDED`. Store timestamps and actor IDs for staff approval.

3. Inventory rule: reduce only after staff approval

Stock is stored in `inventory.available_qty`. `skus.quantity_value` is only pack size, such as 1 kg; it is not stock count.

At staff approval, lock each inventory row and reduce stock atomically. Use an UPDATE condition such as `available_qty >= requested quantity`. If any item lacks enough stock, roll back the entire operation: no bill, no partial deduction, and show the staff a clear failure.

Never reduce `available_qty` when a barcode is scanned, a cart is edited, payment begins, or payment succeeds before staff approval. Update `inventory.last_updated` when stock changes.

The final bill, bill-history record, payment/order state update, and every inventory change must succeed or fail together in one transaction.

4. Barcode scanning and cart behaviour

Fix the second-use scan failure: after one completed customer session, scanning must continue to work for later sessions. Reset camera/scanner listeners, barcode de-duplication flags, cart session state, and any cached processing lock when a checkout ends or the scan screen mounts again.

On each successful scan, look up the active SKU by barcode and verify inventory availability. If the SKU is already in the cart, increase its displayed quantity by one; do not add another visible product line.

Remove the plus (+) quantity button from the cart UI. The only way to increase quantity is to scan the same barcode again. Keep a remove/decrease action only if the product design needs it, and make it clear it does not alter live inventory before approval.

Test unknown barcode, inactive SKU, out-of-stock SKU, repeated scans, rapid scans, navigation away/back, and a completely new checkout after a completed order.

5. Bill visibility and history

Before staff acknowledgement: do not generate a bill PDF/number, do not insert into final bill history, and do not expose a receipt page in the customer app. A pending-payment or waiting-for-staff page is allowed.

After acknowledgement: generate exactly one bill number and final bill, insert it into bill history, and return/show that same approved bill to the customer. The staff portal must display a success/failure result, not silently assume it worked.

If staff rejects or payment expires/fails, no final bill and no inventory deduction may exist. The pending record may be retained for audit, with its outcome state and reason.

6. Database design and migration checklist

Keep `inventory.available_qty` as the authoritative local stock field. Add or confirm durable order/payment/bill tables or fields for status, gateway transaction ID, idempotency key, `staff_approved_by`, `staff_approved_at`, and failure/rejection reason.

Add a bill-items table that records SKU, immutable product label, quantity, price, tax, and totals as they were at approval time. Do not rebuild old bills from current catalog prices.

Create a database migration locally first. It must be repeatable/safe where possible and include indexes/unique constraints for bill number, gateway transaction ID, and idempotency key. Do not apply it to Neon until approval is given.

The existing live database already has `inventory.available_qty` for all 547 SKUs. The implementation issue is when it is updated, not the absence of the column.

7. Duplicate shop database synchronisation

There are two databases: the duplicate database connected to the app and the original shop database. The duplicate is operational; changes made there must be propagated to the original every 30 minutes.

Before implementing, obtain written details for the original database connection, permitted tables, conflict policy, timezone, and recovery owner. Do not guess these. Do not expose either connection string in client apps or source control.

Implement a scheduled server-side job every 30 minutes. Sync from an append-only outbox/change-log created in the same transaction as approved sales and stock/import updates. Mark an outbox event synced only after the original database confirms it.

The job must be idempotent, retry safely, log each run, alert on failures, and never overwrite newer original-database data blindly. Include reconciliation: compare stock/order totals and report mismatches. Define whether original-to-duplicate updates are also required; this is not specified yet.

8. Importing products / increasing stock

Product imports must increase `inventory.available_qty`, not overwrite it accidentally, unless an explicitly approved stock-take/import mode says otherwise. Validate SKU and store, reject negative quantities, record who imported stock and when, and create an outbox event for the 30-minute sync.

Use the same transaction discipline as checkout so the inventory change and its audit/sync record cannot diverge.

9. Acceptance tests the intern must run

- A. Scan the same SKU twice: one cart line with quantity 2; no plus button is visible; stock unchanged before approval.
- B. Finish one checkout, start a second checkout, scan a product: it is added correctly. Repeat after navigating away/back and after denied camera permission recovery if applicable.
- C. Payment received but staff has not approved: customer sees pending state; no final bill/history entry; stock unchanged.
- D. Staff approves: exactly one final bill and history entry exist; customer receives the receipt page; available_qty decreases by the bill quantity.
- E. Staff double-click/retry, duplicate webhook, or network retry: still one bill and one stock deduction only.
- F. Insufficient stock at approval: transaction fails cleanly; no bill/history entry; no SKU stock is changed.
- G. Payment failure, staff rejection, timeout/cancellation: no final bill and no stock reduction.
- H. Import stock: quantity increases correctly and is included in the next synchronisation event.
- I. Trigger/test the 30-minute sync using a safe test mode: approved sales/import events reach original DB once; retry and outage recovery do not duplicate updates; reconciliation is clean.

10. Final test report required from the intern

Send one concise report containing: build/version and environment; database migration names (local only unless approved); each acceptance test with PASS/FAIL, exact steps, expected result, actual result, and evidence; before/after available_qty values for a test SKU; proof that pending payments create no bill and no deduction; proof of idempotency; sync job run IDs and reconciliation result; known defects/risks; and a clear recommendation: READY / NOT READY.

Testing is mandatory after implementation. Any failed critical test (duplicate deduction, premature bill, premature stock reduction, missing second-session scan, or unsafe sync) means NOT READY.

Open decisions for the project owner

Choose the payment gateway and confirm its credentials/webhook environment. Provide the original shop database access and the sync conflict/data-ownership policy. Confirm staff roles allowed to approve/reject. Confirm how refunds and partial fulfilment should restore inventory. Confirm the scheduled-job hosting/monitoring location.